Tutorial 1: ASP.NET Core MVC with CRUD (No Scaffold)

Goal:

Create a web app with full CRUD functionality for the `Student` entity using MVC pattern, EF Core, and MSSQL LocalDB — *manually coded, no scaffolding*.

🛠️ Tools & Technologies

- Visual Studio 2022
- .NET 6 SDK
- SQL Server Express LocalDB
- Entity Framework Core 6 (Code First)

📁 Final Folder Structure

```
StudentManagement/
│
├── Controllers/
│   └── StudentController.cs
│
├── Data/
│   └── ApplicationDbContext.cs
│
├── Models/
│   └── Student.cs
│
├── Views/
│   └── Student/
│       ├── Index.cshtml
│       ├── Details.cshtml
│       ├── Create.cshtml
│       ├── Edit.cshtml
│       └── Delete.cshtml
│
├── appsettings.json
├── Program.cs
```

✅ Step-by-Step Instructions

1️⃣ Create a New MVC Project

- Open **Visual Studio 2022**
- Create New Project → **ASP.NET Core Web App (Model-View-Controller)**
- Name it: `StudentManagement`
- Framework: **.NET 6 (LTS)**
- Authentication: None

- Enable SSL: None

2️⃣ Install EF Core Packages

Open the **Package Manager Console** and run following command or you can use **Nuget Packages** to install with GUI

```
Install-Package Microsoft.EntityFrameworkCore -Version 6.0.28
Install-Package Microsoft.EntityFrameworkCore.SqlServer -Version 6.0.28
Install-Package Microsoft.EntityFrameworkCore.Tools -Version 6.0.28
```

3️⃣ Create the `Student` Model

📄 `Models/Student.cs`

```
using System.ComponentModel.DataAnnotations;

namespace StudentManagement.Models
{
    public class Student
    {
        public int Id { get; set; }

        [Required]
        public string FullName { get; set; }

        [EmailAddress]
        public string Email { get; set; }

        [DataType(DataType.Date)]
        public DateTime DateOfBirth { get; set; }
    }
}
```

4️⃣ Setup EF Core DbContext

📄 `Data/ApplicationDbContext.cs`

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Models;

namespace StudentManagement.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Student> Students { get; set; }
    }
}
```

5️⃣ Configure Connection String

📄 `appsettings.json`

```
"ConnectionStrings": {
  "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=StudentDb;Trusted_Connection=True;"
}
```

6️⃣ Register DbContext in `Program.cs`

📄 `Program.cs` (For .NET 6)

```
using Microsoft.EntityFrameworkCore;
using StudentManagement.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

// Middleware
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
}

app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

// Default route
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Student}/{action=Index}/{id?}");

app.Run();
```

7️⃣ Create the Database

Run these in **Package Manager Console**:

```
Add-Migration InitialCreate
Update-Database
```

8️⃣ Create `StudentController` Manually

📄 `Controllers/StudentController.cs`

```
using Microsoft.AspNetCore.Mvc;
using StudentManagement.Data;
using StudentManagement.Models;
using Microsoft.EntityFrameworkCore;

namespace StudentManagement.Controllers
{
```

```
public class StudentController : Controller
{
    private readonly ApplicationDbContext _context;

    public StudentController(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task<IActionResult> Index()
    {
        return View(await _context.Students.ToListAsync());
    }

    public async Task<IActionResult> Details(int id)
    {
        var student = await _context.Students.FindAsync(id);
        if (student == null) return NotFound();
        return View(student);
    }

    public IActionResult Create() => View();

    [HttpPost]
    public async Task<IActionResult> Create(Student student)
    {
        if (ModelState.IsValid)
        {
            _context.Add(student);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        return View(student);
    }

    public async Task<IActionResult> Edit(int id)
    {
        var student = await _context.Students.FindAsync(id);
        if (student == null) return NotFound();
        return View(student);
    }

    [HttpPost]
    public async Task<IActionResult> Edit(int id, Student student)
    {
        if (id != student.Id) return NotFound();
        if (ModelState.IsValid)
        {
            _context.Update(student);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        return View(student);
    }

    public async Task<IActionResult> Delete(int id)
    {
        var student = await _context.Students.FindAsync(id);
```

```
        if (student == null) return NotFound();
        return View(student);
    }

    [HttpPost, ActionName("Delete")]
    public async Task<IActionResult> DeleteConfirmed(int id)
    {
        var student = await _context.Students.FindAsync(id);
        if (student != null)
        {
            _context.Students.Remove(student);
            await _context.SaveChangesAsync();
        }
        return RedirectToAction(nameof(Index));
    }
  }
}
```

9️⃣ Create Views (Razor)

📁 Create folder: `Views/Student`

📄 `Views/Student/Index.cshtml`

```
@model IEnumerable<StudentManagement.Models.Student>
@{
  ViewData["Title"] = "Students";
}

<h2>Students</h2>
<a href="/Student/Create">+ Add Student</a>
<table>
  <thead>
    <tr>
      <th>Full Name</th>
      <th>Email</th>
      <th>DOB</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var student in Model)
    {
      <tr>
        <td>@student.FullName</td>
        <td>@student.Email</td>
        <td>@student.DateOfBirth.ToShortDateString()</td>
        <td>
          <a href="/Student/Details/@student.Id">View</a> |
          <a href="/Student/Edit/@student.Id">Edit</a> |
          <a href="/Student/Delete/@student.Id">Delete</a>
        </td>
      </tr>
    }
  </tbody>
</table>
```

Views/Student/create.cshtml

```
@model StudentManagement.Models.Student
@{
    ViewData["Title"] = "Create Student";
}

<h2>Create Student</h2>

<form asp-action="Create" method="post">
    <div class="form-group">
        <label for="FullName">Full Name</label>
        <input asp-for="FullName" class="form-control" />
        <span asp-validation-for="FullName" class="text-danger"></span>
    </div>

    <div class="form-group">
        <label for="Email">Email</label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>

    <div class="form-group">
        <label for="DateOfBirth">Date of Birth</label>
        <input asp-for="DateOfBirth" type="date" class="form-control" />
        <span asp-validation-for="DateOfBirth" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Create</button>
</form>

<a href="@Url.Action("Index")">Back to List</a>
```

Views/Student/edit.cshtml

```
@model StudentManagement.Models.Student
@{
    ViewData["Title"] = "Edit Student";
}

<h2>Edit Student</h2>

<form asp-action="Edit" method="post">
    <div class="form-group">
        <label for="FullName">Full Name</label>
        <input asp-for="FullName" class="form-control" />
        <span asp-validation-for="FullName" class="text-danger"></span>
    </div>

    <div class="form-group">
        <label for="Email">Email</label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>

    <div class="form-group">
        <label for="DateOfBirth">Date of Birth</label>
```

```
        <input asp-for="DateOfBirth" type="date" class="form-control" />
        <span asp-validation-for="DateOfBirth" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Save Changes</button>
</form>

<a href="@Url.Action("Index")">Back to List</a>
```

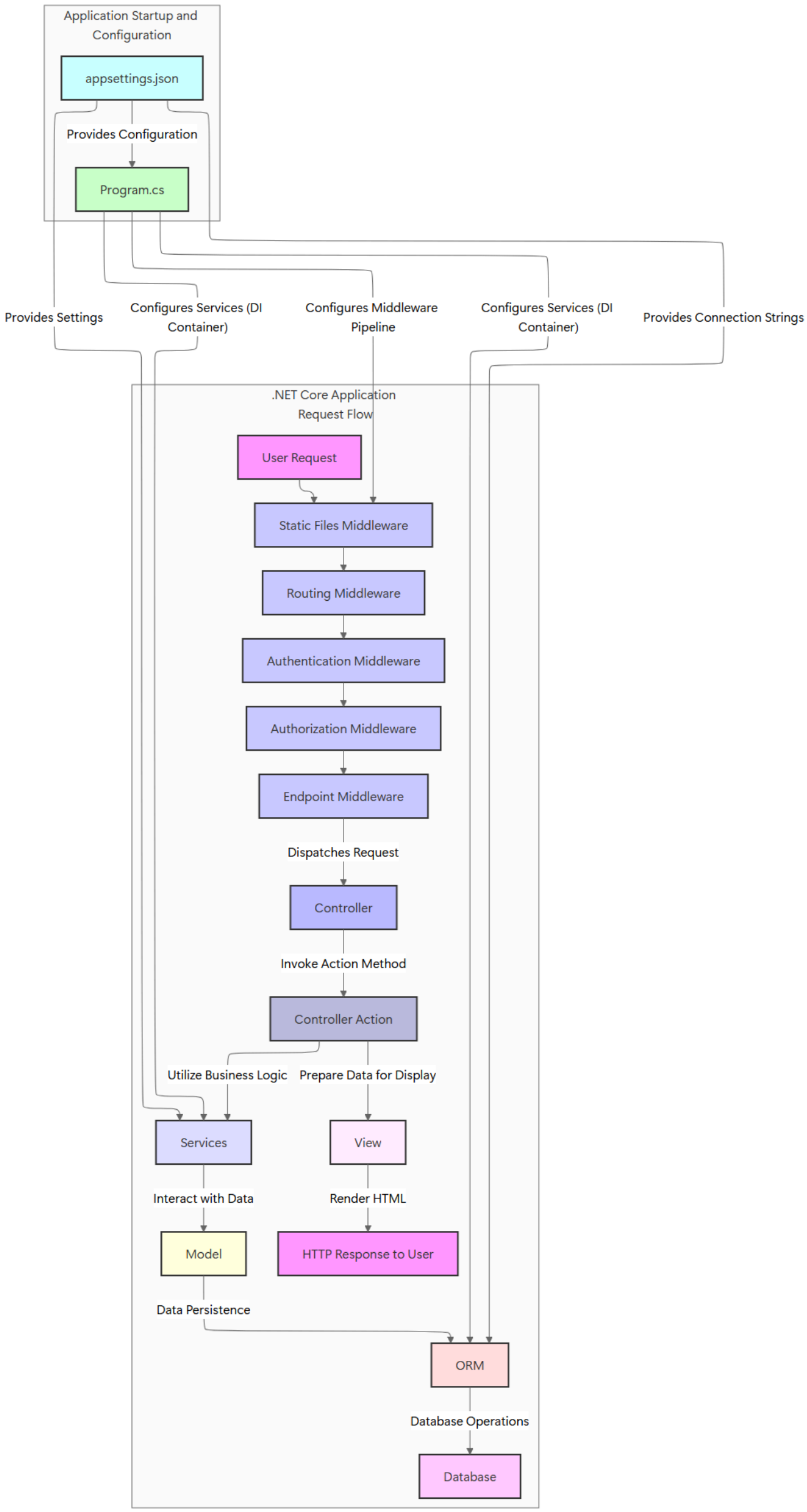
Exercise

- [ ] Complete the `View` and `Delete` pages

✅ Run the App

- Hit **F5** in Visual Studio
- Navigate to `/Student`
- Try all CRUD operations

Pipeline visualization


Application Startup and Configuration
appsettings.json
Provides Configuration
Program.cs
Provides Settings
Configures Services (DI Container)
Configures Middleware Pipeline
Configures Services (DI Container)
Provides Connection Strings
.NET Core Application Request Flow
User Request
Static Files Middleware
Routing Middleware
Authentication Middleware
Authorization Middleware
Endpoint Middleware
Dispatches Request
Controller
Invoke Action Method
Controller Action
Utilize Business Logic
Prepare Data for Display
Services
View
Interact with Data
Render HTML
Model
HTTP Response to User
Data Persistence
ORM
Database Operations
Database